ZACA BUS

Módulo de Taller y conexión entre bases de datos (FDW)

Qué se construyó, cómo funciona por dentro y cómo llevarlo a Linux con otras direcciones IP

Noviembre de 2026

Proyecto de terminal de autobuses · UNACH

Contenido

1. Qué es este documento y la idea en una página
2. Arquitectura
3. Qué se hizo, en orden cronológico
4. La base de datos: objetos y código
5. Cómo funciona el FDW por dentro
6. Por qué la aplicación usa una sola conexión
7. Cómo accede el jefe de taller a la lista de placas
8. Archivos del proyecto Java
9. Códigos de error de la base
10. Cómo probar el flujo completo
11. Pasar a Linux, otras IP, Pgpool y réplica
12. Estado de los scripts y cómo verificarlo
13. Problemas encontrados y cómo se resolvieron
14. Pendientes y recomendaciones
15. Glosario

1. Qué es este documento y la idea en una página

Este documento explica, de principio a fin, todo lo que se construyó en el módulo de **Taller** de Zaca Bus: qué se agregó, cómo funciona por dentro la conexión entre las dos bases de datos (FDW), por qué la aplicación Java usa **una sola conexión**, cómo el jefe de taller llega a ver la lista de placas, y qué habría que cambiar para pasar todo a Linux con direcciones IP distintas.

La idea en pocas líneas

- Existen **dos bases de datos PostgreSQL**: **zacabus** (el sistema principal: rutas, unidades, personal, ventas) y **taller** (el estado mecánico de cada autobús y sus órdenes de reparación).
- La aplicación Java se conecta **únicamente a zacabus**. Nunca abre una conexión a taller.
- Dentro de zacabus, una extensión de PostgreSQL llamada **postgres_fdw** (Foreign Data Wrapper) crea "tablas foráneas": son tablas que parecen locales, pero cuyos datos viven realmente en la base taller. Cuando se consulta o se modifica una tabla foránea, PostgreSQL se conecta a taller por su cuenta.
- Las **placas** de los autobuses las registra el administrador en la pantalla Unidades (tabla viajes.unidad, en zacabus). El jefe de taller las ve porque una vista en zacabus parte de esa lista y le pega, por FDW, el estado que lleva el taller. Un autobús que nunca ha entrado a taller se considera **operativo**.
- Cuando el jefe registra un ingreso, un procedimiento de zacabus crea la orden y cambia el estado del autobús **en la base taller** a través del FDW. A partir de ese momento el despachador ya no puede programar esa unidad: la base de datos lo impide.

> Todo lo que viaja entre las dos bases pasa por nombres (server srv_taller, esquema taller_remoto). Ningún procedimiento, vista ni clase Java contiene una dirección IP. Por eso mudar el sistema a otras máquinas es cambiar la dirección del servidor foráneo y el .env, no reescribir código (sección 11).

2. Arquitectura

2.1 Diagrama general

Figura 1. Quién habla con quién

```
App Java (JavaFX)  -- pantallas: admin, despachador, taquillero, jefe de taller
     |
     |  UNA sola conexion JDBC  (DB_URL del archivo .env)
     v
Pgpool-II  (al final del proyecto; hoy la app apunta directo a zacabus)
     |
     +--> zacabus  (laptop principal)      <-- streaming -->   replica (laptop 2, puerto 5433)
              |
              |   postgres_fdw : servidor foraneo "srv_taller"  (host, port, dbname)
              v
          taller  (maquina virtual, puerto 5432)
```

2.2 Qué vive en cada base

Base	Contiene	Quién la escribe
zacabus	Unidades (placas), personal, cuentas, talleres (catálogo), rutas, viajes programados, ventas, y todas las vistas/procedimientos del módulo taller.	La app Java, a través de procedimientos almacenados.
taller	estado_taller (catálogo de 4 estados), bus_taller (estado actual de cada autobús que ya ha entrado a taller), orden_completa (órdenes de reparación).	Solo zacabus, mediante el FDW (los procedimientos sp_registrar_ingreso y sp_cerrar_orden).

2.3 Por qué dos bases

El proyecto es de bases de datos distribuidas: el taller es un sistema con vida propia (su propio servidor, en una máquina virtual) y el sistema principal solo necesita saber si un autobús está apto o no. El FDW permite que zacabus use los datos del taller como si fueran suyos sin copiarlos: no hay sincronización ni duplicados, siempre se lee el dato vivo.

2.4 Despliegue final planeado

Máquina	Contenido	Puerto
Laptop física 1 (principal)	Base zacabus (escritura)	5432
Laptop física 2	Réplica de zacabus por streaming (solo lectura)	5433
Máquina virtual	Base taller (servidor PostgreSQL independiente, sin réplica)	5432
Delante de las laptops	Pgpool-II: único punto de entrada de la app	9999 (por defecto)

Hoy, durante el desarrollo, las dos bases están en la misma máquina y el mismo servidor PostgreSQL (localhost:5432). Esa es la única diferencia con el despliegue final, y se resuelve con la dirección del servidor foráneo (sección 11).

3. Qué se hizo, en orden cronológico

Cada fase indica qué se pidió, qué se cambió y por qué. Los nombres de archivo están en la sección 8 y los objetos de base de datos en la sección 4.

Fase 0. Revisión previa de los scripts (base de partida)

Antes de escribir código se revisaron el script master y el script de la base taller. Se detectaron las causas por las que un FDW no habría funcionado: no existía ninguna instrucción de FDW (extensión, servidor, mapeo de usuario, importación), el nombre de la base en el script master no coincidía con el real (zacabus), la base taller no tenía esquema ni permisos, el master no consumía el estado del taller en

ninguna parte, y había datos duplicados entre ambas bases (mecánicos). Esa revisión se entregó en un primer documento Word.

Fase 1. Eliminación de las bahías

Decisión del equipo: la capacidad en bahías de un taller deja de existir en **todo** el sistema. Se quitó de:

- El script master: columna capacidad_bahias, su restricción CHECK y el parámetro p_bahias de sp_guardar_taller.
- Java: el campo bahias del record Taller y bahia del record OrdenTaller; la columna "Bahía" de la tabla de órdenes; el campo "Capacidad (Bahías)" del formulario de taller; la columna "Capacidad (Bahías)" de la lista de talleres, que ahora se llama "Autobuses".
- Base real: bloque en CAMBIOS Y ASI.txt con DROP COLUMN y la nueva sp_guardar_taller de 5 parámetros (se hizo DROP PROCEDURE de la firma vieja porque, en PostgreSQL, CREATE OR REPLACE con otros parámetros crea una segunda versión en lugar de reemplazar).

Fase 2. Pantallas del administrador

Las pantallas "Taller" y "Órdenes de taller" del administrador ya existían dibujadas pero sin datos (los DAO devolvían listas vacías). Quedaron conectadas:

- **Talleres**: lista con filtro por ciudad, buscador, alta y edición con sp_guardar_taller, baja y reactivación con sp_cambiar_estado_taller, y asignación del encargado (jefe de taller) con sp_asignar_encargado.
- **Órdenes de taller**: auditoría de solo lectura de todas las órdenes de todos los talleres, con detalle de cada orden.
- **Unidades**: columna nueva "En taller" con el estatus de cada autobús.

Fase 3. Pantallas del jefe de taller (nuevas)

Antes, al iniciar sesión, el jefe de taller aterrizaba en la pantalla del administrador con la barra del administrador. Ahora tiene su propia carpeta de vistas y su propio inicio:

- Barra lateral propia (barra-jefe-taller.fxml) con dos opciones: Órdenes y Flota.
- **Flota**: semáforo de todos los autobuses (estatus, último ingreso, observaciones).
- **Órdenes**: lista de las órdenes de su taller con botón "+ Registrar ingreso" y "Cerrar orden". Los dos formularios (ingreso y cierre) ya existían dibujados; se movieron a la carpeta del jefe y se conectaron.
- El login (LoginController) manda al rol JEFE_TALLER a la vista ordenes-jefe. App.java registra la carpeta nueva jefetaller y module-info.java abre el paquete de controladores nuevo a JavaFX.

Fase 4. Taller asignado en Personal y en Cuentas

Para que un jefe de taller pueda tener cuenta, la base exige que tenga taller asignado (trigger, error Z0025). Se corrigieron varios puntos que hacían parecer que la asignación no existía:

- El formulario de empleado solo mostraba el combo "Taller asignado" para el puesto MECANICO; ahora también para JEFE_TALLER, la lista sale de la base (v_talleres_activos) y lo elegido se guarda. Antes pasaba null siempre y editar a un empleado le borraba el taller.

- La columna "Asignación" de Personal estaba fija en "—". Ahora muestra el taller (mecánico, jefe), la terminal (despachador) o terminal y taquilla (taquillero). La vista v_personal ganó las columnas id_taller, id_terminal e id_taquilla.
- La lista de Cuentas mostraba solo la taquilla y no la sede; ahora muestra "Sede · Taquilla" y, para el jefe, su taller (v_cuentas ganó id_taller y nombre_taller). El formulario de alta de cuenta dejó de decir "Sin sede registrada" fijo y muestra la asignación real.
- La lógica del texto de asignación se unificó en una clase (AsignacionPersonal) que usan Personal y Cuentas, para no repetirla.

Fase 5. La base taller y el FDW

- Se creó la base **taller** con el script TALLER.BD.txt: esquema taller, tablas estado_taller, bus_taller, orden_completa, el índice único que impide dos órdenes abiertas para la misma placa, y la vista v_semaforo_unidades.
- Se creó el **FDW** (fdw_demo.sql): extensión postgres_fdw, servidor srv_taller, mapeo de usuario, esquema taller_remoto e importación de las tablas foráneas. Se verificó con cuatro pruebas en terminal, incluida una que demuestra que un cambio hecho en taller se ve al instante en zacabus.

Fase 6. El flujo de taller en la base de datos (Parte 1)

Con el FDW funcionando se escribió la lógica completa del flujo como un bloque de SQL (CAMBIOS Y ASI.txt, secciones A y B):

- El estado de cada autobús se calcula sobre **toda** la flota activa. Un autobús sin fila en taller cuenta como operativo, así que ya no hay que insertar placas a mano.
- sp_registrar_ingreso y sp_cerrar_orden: meten y sacan un autobús del taller, validando todo dentro de la base.
- Un trigger rechaza programar una salida con una unidad que no esté apta (error Z0005).
- Vistas para las pantallas: flota, unidades disponibles para el despachador, mecánicos del taller, órdenes con nombres.

Fase 7. Conexión de la aplicación Java (Parte 2)

- Jefe de taller: Flota y Órdenes leen la base real; los formularios de ingreso y cierre llaman a los procedimientos y muestran el mensaje exacto de la base si algo falla.
- Despachador: el combo de unidades de "Programar salida" solo ofrece unidades aptas; el error Z0005 se muestra con su mensaje.
- Administrador: la auditoría de órdenes y el conteo de autobuses por taller usan las mismas vistas.

Fase 8. Pendientes cerrados

- Baja y reactivación de talleres (sp_cambiar_estado_taller; se rechaza si el taller tiene personal activo).
- Combo de encargado del taller (vista v_jefes_taller + sp_asignar_encargado, en la misma transacción que el guardado).

- Columna "En taller" en Unidades.

Fase 9. Correcciones menores

- OrdenesTallerController tenía los acentos dañados (por ejemplo "ReparaciÃ³n"); se corrigieron, incluidos los textos que se ven en pantalla.
- Se retiró una propuesta intermedia (una segunda conexión Java a la base taller): no hace falta, el FDW la vuelve innecesaria (sección 6).
- fdw_demo.sql contiene la contraseña de la base; se agregó a .gitignore para que no se suba al repositorio.

4. La base de datos: qué objetos existen y para qué sirve cada uno

4.1 Catálogo de objetos

Objeto	Base	Tipo	Para qué sirve
taller.estado_taller	taller	Tabla	Catálogo: 1 OPERATIVO, 2 MANTENIMIENTO PREVENTIVO, 3 MANTENIMIENTO CORRECTIVO, 4 DE BAJA.
taller.bus_taller	taller	Tabla	Estado actual de cada autobús que ya ha entrado a taller (placa, estado, taller, último ingreso, observaciones). Un autobús sin fila aquí se considera operativo.
taller.orden_completa	taller	Tabla	Órdenes de reparación. Ligada a bus_taller por placa. Guarda además el tipo de mantenimiento con que ingresó (id_estado_ingreso). El índice unq_orden_abierta impide dos órdenes abiertas para la misma placa.
taller.v_semaforo_unidades	taller	Vista	Placa + estatus + apto_para_viajar de lo que hay en bus_taller. Primera vista que se probó por FDW.
srv_taller	zacabus	Servidor foráneo	Dónde está la base taller: host, puerto y nombre de la base. Es la única pieza que lleva una dirección.

Objeto	Base	Tipo	Para qué sirve
USER MAPPING	zacabus	Mapeo	Con qué usuario y contraseña entra zacabus a taller.
taller_remoto.bus_taller, estado_taller, orden_completa, v_semaforo_unidades	zacabus	Tablas foráneas	Espejos de las tablas de taller. Se pueden leer y escribir como si fueran locales.
taller_remoto.orden_ingreso	zacabus	Tabla foránea	Apunta a orden_completa pero sin la columna id_orden; se usa solo para insertar órdenes (explicación en 5.4).
viajes.v_flota_taller	zacabus	Vista	Toda la flota activa con su estatus: unidades del administrador + estado del taller. Sin fila en taller = OPERATIVO.
viajes.v_unidades_disponibles	zacabus	Vista	Unidades que el despachador puede programar (las aptas).
administracion.v_mecanicos_taller	zacabus	Vista	Mecánicos activos y a qué taller pertenecen (para el formulario de ingreso).
viajes.v_ordenes_taller	zacabus	Vista	Órdenes con placa, modelo, taller, mecánico, tipo de servicio y si siguen abiertas.
viajes.sp_registrar_ingreso	zacabus	Procedimiento	Mete un autobús a taller: valida, actualiza bus_taller y crea la orden.
viajes.sp_cerrar_orden	zacabus	Procedimiento	Cierra una orden y deja el autobús operativo o de baja.
viajes.fn_validar_unidad_taller + tigresito_validar_unidad_taller	zacabus	Función + trigger	Rechaza (Z0005) programar una salida con una unidad no apta.
administracion.sp_guardar_taller	zacabus	Procedimiento	Alta y edición de talleres (5 parámetros, sin bahías).
administracion.sp_cambiar_estado_taller	zacabus	Procedimiento	Baja y reactivación de un taller; se rechaza (Z0052) si tiene personal activo.
administracion.sp_asignar_encargado	zacabus	Procedimiento	Asigna un jefe de taller como encargado de un taller.
administracion.v_talleres, v_talleres_activos, v_jefes_taller	zacabus	Vistas	Lista de talleres con mecánicos y encargado; talleres activos para combos; jefes de taller activos.

Objeto	Base	Tipo	Para qué sirve
administracion.v_personal, v_cuentas	zacabus	Vistas (ampliadas)	Ahora traen id_taller, id_terminal, id_taquilla (personal) y id_taller, nombre_taller (cuentas).

4.2 La base taller

TALLER.BD.txt: tablas

```
CREATE TABLE taller.estado_taller(
id_estado INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
nombre_estado TEXT NOT NULL UNIQUE
);



CREATE TABLE taller.bus_taller(
placa TEXT primary key,
id_estado INT NOT NULL DEFAULT 1,
id_taller INT,
ultima_fecha_ingreso TIMESTAMPTZ,
observaciones TEXT,
CONSTRAINT fk_estado_bus FOREIGN KEY (id_estado) REFERENCES taller.estado_taller(id_estado) ON DELETE
RESTRICT ON UPDATE CASCADE
);



CREATE TABLE taller.orden_completa(
id_orden INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
placa TEXT NOT NULL,
id_taller INT NOT NULL,
id_personal INT NOT NULL,
fecha_ingreso TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
fecha_salida TIMESTAMPTZ,
diagnostico TEXT,
costo DECIMAL(10,2) NOT NULL,
CONSTRAINT check_fecha_salida CHECK (fecha_salida IS NULL OR fecha_salida >= fecha_ingreso),
CONSTRAINT check_costo_positivo CHECK (costo >= 0),
CONSTRAINT fk_placa_orden FOREIGN KEY (placa) REFERENCES taller.bus_taller(placa) ON DELETE RESTRICT ON
UPDATE CASCADE
);

CREATE UNIQUE INDEX unq_orden_abierta ON taller.orden_completa (placa) WHERE fecha_salida IS NULL;


--
////////////////////////////////////////////////////////////////////////////////////////////////////////
//////
```

Columna agregada después (CAMBIOS Y ASI.txt, parte A, conectado a taller)

```
-- A1) La orden guarda el tipo de mantenimiento con que ingreso (2 preventivo, 3 correctivo). Hoy no se
guardaba.
ALTER TABLE taller.orden_completa
  ADD COLUMN id_estado_ingreso INT REFERENCES taller.estado_taller(id_estado) ON DELETE RESTRICT ON UPDATE
CASCADE;
```

El índice parcial unq_orden_abierta es una regla de negocio en la base: solo puede existir **una orden con fecha_salida vacía por placa**. Aunque la aplicación fallara, la base no permitiría dos órdenes abiertas para el mismo autobús.

4.3 El puente: servidor, mapeo y tablas foráneas

fdw_demo.sql, parte 2 (contraseña enmascarada)

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

CREATE SERVER srv_taller FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host 'localhost', port '5432', dbname 'taller');

CREATE USER MAPPING FOR CURRENT_USER SERVER srv_taller
  OPTIONS (user 'postgres', password 'TU_PASSWORD');

CREATE SCHEMA taller_remoto;

IMPORT FOREIGN SCHEMA taller LIMIT TO (v_semaforo_unidades)
  FROM SERVER srv_taller INTO taller_remoto;
```

CAMBIOS Y ASI.txt, B1 y B2: lo que se importa y la tabla foránea solo para insertar órdenes

```
-- B1) Tablas del taller que zacabus lee y escribe (solo estas tres; el FDW ya existe: srv_taller).
--     Se importan DESPUES de A1 para que orden_completa traiga la columna nueva.
IMPORT FOREIGN SCHEMA taller LIMIT TO (bus_taller, orden_completa, estado_taller)
  FROM SERVER srv_taller INTO taller_remoto;

-- B2) Tabla solo para INSERTAR ordenes. No incluye id_orden a proposito: postgres_fdw manda NULL en las
columnas
--     que no se escriben, y eso choca con la identidad GENERATED ALWAYS de la tabla remota. Sin esa
columna, el
--     taller genera el numero de orden solo.
CREATE FOREIGN TABLE taller_remoto.orden_ingreso(
placa TEXT NOT NULL,
id_taller INT NOT NULL,
id_personal INT NOT NULL,
fecha_ingreso TIMESTAMPTZ NOT NULL,
diagnostico TEXT,
costo DECIMAL(10,2) NOT NULL,
id_estado_ingreso INT
) SERVER srv_taller OPTIONS (schema_name 'taller', table_name 'orden_completa');
```

4.4 Vistas del semáforo

B3 y B4

```
-- B3) Semaforo de TODA la flota activa. Sin fila en el taller = operativa.
CREATE OR REPLACE VIEW viajes.v_flota_taller AS
SELECT
u.placa,
u.modelo,
COALESCE(e.nombre_estado, 'OPERATIVO') AS estatus,
COALESCE(b.id_estado, 1) = 1 AS apto_para_viajar,
b.ultima_fecha_ingreso,
b.observaciones,
u.id_unidad,
b.id_taller
FROM viajes.unidad u
LEFT JOIN taller_remoto.bus_taller b ON b.placa = u.placa
LEFT JOIN taller_remoto.estado_taller e ON e.id_estado = b.id_estado
WHERE u.activo
```

```
ORDER BY u.placa;



-- B4) Unidades que el despachador puede programar (mismas columnas que la vista anterior)
CREATE OR REPLACE VIEW viajes.v_unidades_disponibles AS
SELECT u.id_unidad, u.id_tipo_bus, u.placa, u.modelo, u.anio, u.kilometraje, u.activo
FROM viajes.unidad u
INNER JOIN viajes.v_flota_taller f ON f.id_unidad = u.id_unidad
WHERE f.apto_para_viajar;
```

Detalle importante de B3: se usa **LEFT JOIN**. Si un autobús no tiene fila en bus_taller, b.id_estado es NULL; COALESCE(b.id_estado, 1) lo convierte en estado 1 (operativo). Gracias a eso, una unidad recién registrada por el administrador aparece de inmediato en la pantalla Flota del jefe y en el combo del despachador, sin que nadie la capture en el taller.

B5 y B6: mecánicos y órdenes

```
-- B5) Mecanicos que el jefe puede elegir (los de su taller)
CREATE OR REPLACE VIEW administracion.v_mecanicos_taller AS
SELECT
p.id_personal,
p.id_taller,
p.nombre_personal || ' ' || p.ap_paterno || COALESCE(' ' || p.ap_materno, '') AS nombre_completo
FROM administracion.personal p
WHERE p.id_puesto = 4 AND p.activo AND p.id_taller IS NOT NULL
ORDER BY p.ap_paterno, p.ap_materno, p.nombre_personal;



-- B6) Ordenes con nombres (las usan la lista del jefe y la auditoria del admin)
CREATE OR REPLACE VIEW viajes.v_ordenes_taller AS
SELECT
o.id_orden,
o.placa,
u.modelo,
o.id_taller,
t.nombre_taller,
o.id_personal,
p.nombre_personal || ' ' || p.ap_paterno AS mecanico,
o.fecha_ingreso,
o.fecha_salida,
o.diagnostico,
o.costo,
CASE o.id_estado_ingreso WHEN 2 THEN 'Preventivo' WHEN 3 THEN 'Correctivo' ELSE '—' END AS tipo_servicio,
o.fecha_salida IS NULL AS abierta
FROM taller_remoto.orden_completa o
LEFT JOIN viajes.unidad u ON u.placa = o.placa
LEFT JOIN administracion.taller t ON t.id_taller = o.id_taller
LEFT JOIN administracion.personal p ON p.id_personal = o.id_personal
ORDER BY o.fecha_ingreso DESC;
```

4.5 Procedimientos del flujo

B7: sp_registrar_ingreso

```
-- B7) Meter un bus a taller. p_tipo: 2 preventivo, 3 correctivo.
--     Z0053 unidad no existe o inactiva | Z0054 ya tiene orden abierta | Z0055 mecanico invalido
--     Z0056 tipo invalido | Z0058 costo invalido | Z0050 taller no existe o inactivo
CREATE OR REPLACE PROCEDURE viajes.sp_registrar_ingreso(p_placa TEXT, p_id_taller INT, p_id_personal INT,
p_tipo INT,
    p_fecha_ingreso TIMESTAMPTZ, p_diagnostico TEXT, p_costo NUMERIC, p_observaciones TEXT)
LANGUAGE plpgsql SECURITY DEFINER SET search_path = pg_catalog, pg_temp AS $$
DECLARE
  v_fecha TIMESTAMPTZ := COALESCE(p_fecha_ingreso, now());
```

```
BEGIN
  IF p_tipo IS NULL OR p_tipo NOT IN (2, 3) THEN
    RAISE EXCEPTION 'Elija si el mantenimiento es preventivo o correctivo.' USING ERRCODE = 'Z0056';
  END IF;
  IF COALESCE(p_costo, -1) < 0 THEN
    RAISE EXCEPTION 'El costo no puede ser negativo.' USING ERRCODE = 'Z0058';
  END IF;
  IF NOT EXISTS (SELECT 1 FROM viajes.unidad WHERE placa = p_placa AND activo) THEN
    RAISE EXCEPTION 'La unidad no existe o está inactiva.' USING ERRCODE = 'Z0053';
  END IF;
  IF NOT EXISTS (SELECT 1 FROM administracion.taller WHERE id_taller = p_id_taller AND activo) THEN
    RAISE EXCEPTION 'El taller no existe o está inactivo.' USING ERRCODE = 'Z0050';
  END IF;
  IF NOT EXISTS (SELECT 1 FROM administracion.personal
                 WHERE id_personal = p_id_personal AND id_puesto = 4 AND activo AND id_taller =
p_id_taller) THEN
    RAISE EXCEPTION 'El mecánico debe ser personal activo de este taller.' USING ERRCODE = 'Z0055';
  END IF;
  IF EXISTS (SELECT 1 FROM taller_remoto.orden_completa WHERE placa = p_placa AND fecha_salida IS NULL)
THEN
    RAISE EXCEPTION 'Esa unidad ya tiene una orden abierta.' USING ERRCODE = 'Z0054';
  END IF;

  -- Primero el estado del bus (la orden remota depende de que exista su fila), luego la orden.
  UPDATE taller_remoto.bus_taller
  SET id_estado = p_tipo, id_taller = p_id_taller, ultima_fecha_ingreso = v_fecha,
      observaciones = NULLIF(btrim(p_observaciones), '')
  WHERE placa = p_placa;
  IF NOT FOUND THEN
    INSERT INTO taller_remoto.bus_taller (placa, id_estado, id_taller, ultima_fecha_ingreso,
observaciones)
    VALUES (p_placa, p_tipo, p_id_taller, v_fecha, NULLIF(btrim(p_observaciones), ''));
  END IF;

  INSERT INTO taller_remoto.orden_ingreso (placa, id_taller, id_personal, fecha_ingreso, diagnostico,
costo, id_estado_ingreso)
  VALUES (p_placa, p_id_taller, p_id_personal, v_fecha, NULLIF(btrim(p_diagnostico), ''), p_costo,
p_tipo);
END $$;
```

Orden de las operaciones dentro del procedimiento, y por qué: (1) se validan tipo, costo, unidad, taller, mecánico y que no haya orden abierta, todo contra datos locales salvo la orden abierta; (2) se actualiza o inserta la fila de bus_taller (el estado del autobús); (3) se inserta la orden. La fila de bus_taller va primero porque la orden remota tiene una llave foránea hacia ella. Como el UPDATE sobre una tabla foránea no admite ON CONFLICT, el "upsert" se hace con UPDATE y, si no encontró fila (IF NOT FOUND), INSERT.

B8: sp_cerrar_orden

```
-- B8) Sacar un bus de taller. p_estado_final: 1 operativo, 4 de baja. Solo cierra ordenes abiertas DE ESE
TALLER.
--     Z0057 la orden no existe, ya esta cerrada o es de otro taller | Z0058 dato invalido
CREATE OR REPLACE PROCEDURE viajes.sp_cerrar_orden(p_id_orden INT, p_id_taller INT, p_fecha_salida
TIMESTAMPTZ,
    p_diagnostico TEXT, p_costo NUMERIC, p_estado_final INT)
LANGUAGE plpgsql SECURITY DEFINER SET search_path = pg_catalog, pg_temp AS $$
DECLARE
  v_placa TEXT;
  v_ingreso TIMESTAMPTZ;
  v_salida TIMESTAMPTZ := COALESCE(p_fecha_salida, now());
BEGIN
  SELECT placa, fecha_ingreso INTO v_placa, v_ingreso
  FROM taller_remoto.orden_completa
  WHERE id_orden = p_id_orden AND id_taller = p_id_taller AND fecha_salida IS NULL;
  IF NOT FOUND THEN
```

```
    RAISE EXCEPTION 'La orden no existe, ya está cerrada o es de otro taller.' USING ERRCODE = 'Z0057';
  END IF;
  IF p_estado_final IS NULL OR p_estado_final NOT IN (1, 4) THEN
    RAISE EXCEPTION 'Elija si el bus sale operativo o de baja.' USING ERRCODE = 'Z0058';
  END IF;
  IF COALESCE(p_costo, -1) < 0 THEN
    RAISE EXCEPTION 'El costo no puede ser negativo.' USING ERRCODE = 'Z0058';
  END IF;
  IF v_salida < v_ingreso THEN
    RAISE EXCEPTION 'La salida no puede ser anterior al ingreso.' USING ERRCODE = 'Z0058';
  END IF;

  UPDATE taller_remoto.orden_completa
  SET fecha_salida = v_salida, diagnostico = NULLIF(btrim(p_diagnostico), ''), costo = p_costo
  WHERE id_orden = p_id_orden;

  UPDATE taller_remoto.bus_taller SET id_estado = p_estado_final WHERE placa = v_placa;
END $$;
```

sp_cerrar_orden exige el id del taller: un jefe solo puede cerrar órdenes **de su taller** (si la orden es de otro taller, responde Z0057). Una vez cerrada la orden, el autobús queda en el estado final elegido: 1 operativo o 4 de baja.

B9: la regla que protege al despacho

```
-- B9) La base rechaza programar una unidad que no esta apta (en taller o de baja).
--     Si el taller no responde, falla la consulta y NO se programa: es lo mas seguro.
CREATE OR REPLACE FUNCTION viajes.fn_validar_unidad_taller()
RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = pg_catalog, pg_temp AS $$
BEGIN
  IF NEW.id_unidad IS NOT NULL AND NOT EXISTS (
       SELECT 1 FROM viajes.v_flota_taller WHERE id_unidad = NEW.id_unidad AND apto_para_viajar) THEN
    RAISE EXCEPTION 'La unidad está en taller, de baja o inactiva: no se puede asignar a una salida.'
USING ERRCODE = 'Z0005';
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER tigresito_validar_unidad_taller
BEFORE INSERT OR UPDATE OF id_unidad ON viajes.viaje_programado
FOR EACH ROW EXECUTE FUNCTION viajes.fn_validar_unidad_taller();
```

> Este trigger es la regla definitiva. Que el combo del despachador solo muestre unidades aptas es una ayuda visual; la garantía real es que, aunque alguien intente asignar la unidad por otro camino, la base responde con el error Z0005 y no guarda nada. Además, si la base taller no responde, la consulta falla y la salida **no** se programa: es la opción más segura.

4.6 Vistas ampliadas de Personal y Cuentas

v_personal

```
CREATE OR REPLACE VIEW administracion.v_personal AS
SELECT
p.id_personal,
pu.id_puesto,
pu.nombre_puesto,
p.nombre_personal,
p.ap_paterno,
p.ap_materno,
p.curp,
p.telefono,
p.licencia,
```

```
p.vigencia,
EXISTS (SELECT 1 FROM administracion.personal_app pa WHERE pa.id_personal = p.id_personal) AS
tiene_cuenta,
p.id_taller,
p.id_terminal,
p.id_taquilla
FROM administracion.personal p
INNER JOIN administracion.puesto pu ON pu.id_puesto = p.id_puesto
ORDER BY p.ap_paterno, p.ap_materno, p.nombre_personal;
```

v_cuentas

```
CREATE OR REPLACE VIEW administracion.v_cuentas AS
SELECT
pa.id_app,
pa.id_personal,
pa.username,
p.nombre_personal || ' ' || p.ap_paterno || COALESCE(' ' || p.ap_materno, '') AS nombre_completo,
r.id_rol,
r.nombre_rol,
t.id_terminal,
t.nombre_terminal,
q.id_taquilla,
q.identificador,
pa.activo,
tl.id_taller,
tl.nombre_taller
FROM administracion.personal_app pa
INNER JOIN administracion.personal p ON p.id_personal = pa.id_personal
INNER JOIN administracion.rol r ON r.id_rol = pa.id_rol
LEFT JOIN administracion.terminal t ON t.id_terminal = p.id_terminal
LEFT JOIN administracion.taquilla q ON q.id_taquilla = p.id_taquilla
LEFT JOIN administracion.taller tl ON tl.id_taller = p.id_taller
ORDER BY pa.username;
```

En PostgreSQL, CREATE OR REPLACE VIEW solo permite **agregar columnas al final** de la vista; por eso id_taller, id_terminal, id_taquilla y nombre_taller están al final.

5. Cómo funciona el FDW por dentro

5.1 Las cinco piezas

1. **Extensión** postgres_fdw: el código que sabe hablar con otro PostgreSQL. Se instala una vez por base con CREATE EXTENSION.
2. **Servidor foráneo** (srv_taller): una ficha con host, port y dbname del otro PostgreSQL.
3. **Mapeo de usuario** (USER MAPPING): con qué rol y contraseña de la otra base entra el usuario local.
4. **Tabla foránea** (CREATE FOREIGN TABLE o IMPORT FOREIGN SCHEMA): una tabla que existe en zacabus solo como definición; no guarda filas. Dice "para leer esto, pregúntale a srv_taller por taller.bus_taller".
5. **Esquema taller_remoto**: carpeta de zacabus donde se agrupan las tablas foráneas, para distinguirlas de las locales.

5.2 Qué pasa cuando se lee (la Flota del jefe)

1. Java manda por JDBC: SELECT ... FROM viajes.v_flota_taller.
2. PostgreSQL expande la vista: une viajes.unidad (local) con taller_remoto.bus_taller y taller_remoto.estado_taller (foráneas).
3. Para las tablas foráneas, postgres_fdw abre (o reutiliza) una conexión desde el proceso de zacabus hacia srv_taller, usando el mapeo de usuario, y le envía un SELECT normal sobre taller.bus_taller y taller.estado_taller. Las condiciones del WHERE que se puedan, se envían al otro lado para traer menos filas.
4. taller responde con sus filas; zacabus las junta (JOIN) con las unidades locales.
5. Java recibe un único resultado. Nunca supo que una parte venía de otra base.

La conexión de zacabus hacia taller se mantiene abierta mientras dure la sesión de quien consulta; la primera consulta tarda un poco más porque debe conectarse.

5.3 Qué pasa cuando se escribe (registrar un ingreso)

postgres_fdw también permite INSERT, UPDATE y DELETE en tablas foráneas. Cuando sp_registrar_ingreso hace UPDATE taller_remoto.bus_taller ..., zacabus abre una transacción en taller y le envía el UPDATE. Al terminar el procedimiento, la transacción local se confirma y justo antes se confirma la remota.

> Límite que conviene conocer: no existe confirmación en dos fases entre las dos bases. En el caso extremo de que taller confirmara y justo después fallara la confirmación local de zacabus, quedarían inconsistentes. Los procedimientos están ordenados (primero el estado del bus, luego la orden) y validan todo antes de escribir para que esto sea muy improbable; para este proyecto es un riesgo aceptado.

5.4 Por qué existe taller_remoto.orden_ingreso

La columna id_orden de la tabla remota es una identidad GENERATED ALWAYS: la base taller genera el número sola y no acepta que se lo manden. Pero cuando se inserta en una tabla foránea, postgres_fdw envía **todas** las columnas de la tabla foránea, y a las que no se escribieron les manda NULL. Mandar NULL a una identidad GENERATED ALWAYS da error. La solución fue definir una segunda tabla foránea (orden_ingreso) que apunta a la misma tabla pero **sin la columna id_orden**. Al insertar por ahí, taller genera el número de orden por su cuenta. Para leer se usa taller_remoto.orden_completa, que sí incluye id_orden.

5.5 Qué pasa si taller no responde

- Las pantallas que leen (Flota, Órdenes, Unidades, Talleres) muestran el error en una franja roja y no se cuelgan.
- Programar salida **falla** con un mensaje y no programa nada, porque el trigger necesita consultar el estado de la unidad.
- Recomendación para el despliegue: ALTER SERVER srv_taller OPTIONS (ADD connect_timeout '3'); así una máquina virtual apagada falla en 3 segundos y no deja la pantalla esperando.

5.6 Limitaciones del FDW que afectan el diseño

Limitación	Consecuencia en este proyecto
Un procedimiento (CALL) no se puede ejecutar de forma remota.	sp_registrar_ingreso y sp_cerrar_orden viven en zacabus y escriben en las tablas foráneas; no existen procedimientos en taller para esto.
No hay ON CONFLICT sobre tablas foráneas.	El alta/actualización de bus_taller se hace con UPDATE y, si no encontró fila, INSERT.
Una tabla foránea es una "foto" de la estructura al importarla.	Si se agrega una columna en taller (como id_estado_ingreso), hay que importar de nuevo la tabla. Por eso la parte A se corre antes que la B.
No hay transacción distribuida.	Ver la nota de 5.3.
Cada consulta foránea cuesta una ida y vuelta por la red.	En una sola máquina es inapreciable; en el despliegue con la VM conviene mantener las vistas simples, como están.

6. Por qué la aplicación usa una sola conexión

La clase ConexionBD abre una conexión JDBC y todas las pantallas la comparten. No existe una segunda conexión para el taller; sería innecesaria porque el FDW ya la sustituye.

util/ConexionBD.java

```
/** Si Pgpool reinició o la conexión se cayó, abre una nueva. */
public synchronized Connection getConexion() throws SQLException {
    if (conexion == null || !conexion.isValid(2)) {
        conexion = DriverManager.getConnection(
                config.getProperty("DB_URL"),
                config.getProperty("DB_USER"),
                config.getProperty("DB_PASSWORD"));
    }
    return conexion;
}
```

Los datos de conexión salen del archivo .env que está junto a la aplicación:

zaca_bus/.env (ejemplo del desarrollo actual)

```
DB_URL=jdbc:postgresql://localhost:5432/zacabus
DB_USER=postgres
DB_PASSWORD=****
```

El archivo .env.example del proyecto trae como ejemplo el puerto **9999** (Pgpool-II). En el despliegue final DB_URL apuntará a Pgpool; en desarrollo apunta directo a PostgreSQL (5432).

Qué ve Java y qué ve la base

Capa	Qué ve
Java (DAO)	Una conexión a zacabus. Lanza un SELECT sobre una vista o un CALL a un procedimiento. Recibe filas o un error SQL con su código (por ejemplo Z0054).

Capa	Qué ve
zacabus	Resuelve la vista: parte la consulta en una parte local y una parte foránea.
postgres_fdw	Abre una conexión interna hacia taller, envía lo que corresponde y devuelve el resultado.
taller	Un PostgreSQL normal que recibe consultas de un usuario (el del mapeo). No sabe que detrás hay una aplicación Java.

Ventajas: la aplicación no necesita las credenciales ni la dirección del taller; las reglas están en la base (vistas, procedimientos, trigger) y funcionan igual sin importar desde dónde se llamen; y cambiar de máquina no toca el código Java.

7. Cómo accede el jefe de taller a la lista de placas

7.1 En texto general

El jefe de taller **no consulta la base del taller**. Consulta una vista de zacabus. Esa vista arranca de la lista de **unidades que registró el administrador** (cada unidad tiene su placa) y le agrega, por FDW, lo que el taller sabe de cada una: su estatus, su último ingreso y sus observaciones. Si el taller no sabe nada de una unidad (nunca ha entrado), se muestra como operativa. Por eso las placas "las tiene el administrador" y el jefe las ve sin que nadie las capture dos veces.

Figura 2. De dónde sale cada placa

```
Admin registra una unidad (pantalla Unidades)
        |  sp_guardar_unidad
        v
viajes.unidad  (zacabus)   <-- aqui nace la placa
        |
        |  LEFT JOIN por placa
        v
viajes.v_flota_taller  ---->  taller_remoto.bus_taller   (FDW -> taller.bus_taller)
        |                ---->  taller_remoto.estado_taller (FDW -> taller.estado_taller)
        v
TallerJefeDAO.flota()  -->  FlotaTallerController  -->  tabla "Flota" del jefe
```

7.2 Paso a paso, con el código

Paso 1. El inicio de sesión decide a dónde va el jefe y a qué taller pertenece

controller/LoginController.java

```
private static final Map<String, String> INICIO = Map.of(
        "ADMINISTRADOR", "rutas",
        "DESPACHADOR", "programar-salida",
        "TAQUILLERO", "abrir-turno",
        "JEFE_TALLER", "ordenes-jefe");
```

LoginController valida usuario y contraseña con CuentaDAO.autenticar, que termina leyendo la vista v_cuentas. Esa vista ahora incluye el taller del empleado (id_taller, nombre_taller), así que el objeto Cuenta guardado en la sesión ya trae el taller del jefe.

model/Cuenta.java

```
/** `idApp` es 0 mientras no se ha guardado. `sede`, `taquilla` y `taller` (jefe de taller) pueden ser
null; la taquilla solo aplica a taquilleros. */
public record Cuenta(int idApp, int idPersonal, String username, String nombreCompleto, Catalogo rol,
        Catalogo sede, Catalogo taquilla, boolean activa, Catalogo taller) {
}
```

Paso 2. El botón "Flota" abre la pantalla

Los botones de la barra lateral llevan como id el nombre de la vista (id="flota-taller"). VistaController.irA llama a App.irA, que busca ese nombre dentro de las carpetas de vistas (admin, despachador, taquillero, jefetaller, comun) y carga flota-taller.fxml.

Paso 3. La pantalla pide la lista

controller/jefetaller/FlotaTallerController.java

```
@FXML
private void initialize() {
    colUnidad.setCellValueFactory(c -> texto(c.getValue().placa() + "  " + c.getValue().modelo()));
    colEstatus.setCellValueFactory(c -> texto(c.getValue().estatus()));
    colEstatus.setCellFactory(c -> new TableCell<>() {
        @Override
        protected void updateItem(String estatus, boolean vacia) {
            super.updateItem(estatus, vacia);
            if (vacia) {
                setGraphic(null);
                return;
            }
            BusTaller bus = getTableView().getItems().get(getIndex());
            Label chip = new Label(estatus);
            chip.getStyleClass().addAll("chip", bus.aptoParaViajar() ? "chip-ok" : "chip-aviso");
            setGraphic(chip);
        }
    });
    colIngreso.setCellValueFactory(c -> texto(c.getValue().ultimoIngreso() == null ? "—" :
c.getValue().ultimoIngreso()));
    colObservaciones.setCellValueFactory(c -> texto(c.getValue().observaciones() == null ? "—" :
c.getValue().observaciones()));
    List<BusTaller> flota = List.of();
    try {
        flota = dao.flota();
    } catch (SQLException e) {
        Mensaje.error(lblError, "No se pudo cargar la flota: " + e.getMessage());
    }
    tabla.setItems(Tablas.filtrar(FXCollections.observableArrayList(flota), txtBuscar,
            b -> b.placa() + " " + b.modelo() + " " + b.estatus()));
}
```

Si la consulta falla (por ejemplo, la base taller está apagada), el error aparece en la franja roja de la pantalla y la tabla queda vacía.

Paso 4. El DAO consulta la vista

dao/jefetaller/TallerJefeDAO.java

```
/** Semáforo de la flota. */
public List<BusTaller> flota() throws SQLException {
    List<BusTaller> flota = new ArrayList<>();
    try (Statement st = ConexionBD.getInstancia().getConexion().createStatement();
            ResultSet rs = st.executeQuery("""
                    SELECT placa, modelo, estatus, apto_para_viajar, ultima_fecha_ingreso, observaciones
                    FROM viajes.v_flota_taller""")) {
        while (rs.next()) {
```

```
            OffsetDateTime ingreso = rs.getObject(5, OffsetDateTime.class);
            flota.add(new BusTaller(rs.getString(1), rs.getString(2), rs.getString(3), rs.getBoolean(4),
                    ingreso == null ? null : Formato.fechaHora(ingreso.atZoneSameInstant(Formato.ZONA)),
                    rs.getString(6)));
        }
    }
    return flota;
}
```

Esta es toda la "magia": un SELECT común sobre viajes.v_flota_taller. Que dentro de la vista haya tablas foráneas no se nota desde Java.

Paso 5. La vista junta las dos bases

viajes.v_flota_taller

```
-- B3) Semaforo de TODA la flota activa. Sin fila en el taller = operativa.
CREATE OR REPLACE VIEW viajes.v_flota_taller AS
SELECT
u.placa,
u.modelo,
COALESCE(e.nombre_estado, 'OPERATIVO') AS estatus,
COALESCE(b.id_estado, 1) = 1 AS apto_para_viajar,
b.ultima_fecha_ingreso,
b.observaciones,
u.id_unidad,
b.id_taller
FROM viajes.unidad u
LEFT JOIN taller_remoto.bus_taller b ON b.placa = u.placa
LEFT JOIN taller_remoto.estado_taller e ON e.id_estado = b.id_estado
WHERE u.activo
ORDER BY u.placa;
```

Paso 6. Cómo se pinta

La columna Estatus muestra una etiqueta de color: verde si apto_para_viajar es verdadero (operativo) y ámbar si no lo es (mantenimiento o de baja). El buscador filtra por placa, modelo o estatus.

7.3 Las otras listas que usa el jefe

Unidades que pueden ingresar (formulario "Registrar ingreso")

TallerJefeDAO.unidadesLibres()

```
/** Placas de las unidades que pueden ingresar (aptas: sin orden abierta y no de baja). */
public List<String> unidadesLibres() throws SQLException {
    List<String> placas = new ArrayList<>();
    try (Statement st = ConexionBD.getInstancia().getConexion().createStatement();
            ResultSet rs = st.executeQuery("SELECT placa FROM viajes.v_unidades_disponibles ORDER BY
placa")) {
        while (rs.next()) {
            placas.add(rs.getString(1));
        }
    }
    return placas;
}
```

Lee v_unidades_disponibles: solo las unidades aptas (si ya está en taller o de baja, no aparece para ingresar otra vez).

Mecánicos del taller del jefe

TallerJefeDAO.mecanicos()

```
/** Mecánicos activos del taller. */
public List<Catalogo> mecanicos(int idTaller) throws SQLException {
    List<Catalogo> mecanicos = new ArrayList<>();
    try (PreparedStatement ps = ConexionBD.getInstancia().getConexion().prepareStatement(
            "SELECT id_personal, nombre_completo FROM administracion.v_mecanicos_taller WHERE id_taller =
?")) {
        ps.setInt(1, idTaller);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                mecanicos.add(new Catalogo(rs.getInt(1), rs.getString(2)));
            }
        }
    }
    return mecanicos;
}
```

El taller del jefe sale de su cuenta

OrdenesJefeController.idTaller()

```
/** El taller del jefe sale de su cuenta; sin taller no hay nada que mostrar ni registrar. */
private static Integer idTaller() {
    Catalogo taller = Sesion.actual() == null ? null : Sesion.actual().taller();
    return taller == null ? null : taller.id();
}
```

Todas las operaciones del jefe usan este número: lista solo sus órdenes y lo manda como parámetro a los procedimientos, que validan que el mecánico sea de ese taller y que la orden a cerrar le pertenezca.

7.4 Recorrido completo de un ingreso

1. El jefe pulsa "+ Registrar ingreso". OrdenesJefeController.nuevoIngreso abre orden-ingreso-form.fxml y llama a prepararIngreso, que llena los combos (unidades libres, mecánicos del taller) y precarga fecha y hora.
2. Al elegir una unidad, avisarSalidas cuenta sus salidas programadas a futuro y, si hay, muestra el aviso "el despacho deberá cambiarles la unidad".
3. Al pulsar "Registrar ingreso", registrarIngreso valida los campos (unidad, mecánico, tipo, hora en formato hh:mm, costo numérico) y llama a OrdenTallerDAO.registrarIngreso.
4. El DAO ejecuta CALL viajes.sp_registrar_ingreso(...) dentro de una transacción.
5. El procedimiento valida, actualiza bus_taller por FDW y crea la orden en taller.orden_completa por FDW.
6. Si todo sale bien, la ventana se cierra y la lista se recarga: aparece la orden "En Reparación". Si la base rechaza algo (Z0053 a Z0058), el mensaje exacto se muestra en el formulario.

OrdenesJefeController.registrarIngreso()

```
@FXML
private void registrarIngreso() {
    try {
        if (cbUnidad.getValue() == null || cbMecanico.getValue() == null || cbTipo.getValue() == null) {
            throw new IllegalArgumentException("Elija la unidad, el mecánico y el tipo de
mantenimiento.");
        }
        ZonedDateTime ingreso = fechaHora(dpIngreso, txtHoraIngreso);
```

```
        ordenDAO.registrarIngreso(cbUnidad.getValue(), idTaller(), cbMecanico.getValue().id(),
                "Preventivo".equals(cbTipo.getValue()) ? 2 : 3, ingreso, txtDiagnostico.getText().trim(),
costo(),
                txtObservaciones.getText().trim());
        cerrar();
    } catch (IllegalArgumentException e) {
        Mensaje.error(lblError, e.getMessage());
    } catch (SQLException e) {
        Mensaje.error(lblError, mensaje(e));
    }
}
```

OrdenTallerDAO.registrarIngreso()

```
/**
 * Mete un autobús a taller. `tipo`: 2 preventivo, 3 correctivo. Z0050 taller inactivo; Z0053 unidad
inválida;
 * Z0054 ya tiene orden abierta; Z0055 mecánico que no es del taller; Z0056 tipo inválido; Z0058 costo
inválido.
 */
public void registrarIngreso(String placa, int idTaller, int idMecanico, int tipo, ZonedDateTime ingreso,
        String diagnostico, BigDecimal costo, String observaciones) throws SQLException {
    Sql.transaccion(con -> Sql.ejecutar(con, "CALL viajes.sp_registrar_ingreso(?, ?, ?, ?, ?, ?, ?, ?)",
            placa, idTaller, idMecanico, tipo, ingreso.toOffsetDateTime(), diagnostico, costo,
observaciones));
}
```

7.5 Qué cambia en el resto de pantallas por ese ingreso

- **Despachador**: el combo de Programar salida deja de ofrecer la unidad. El combo lee v_unidades_disponibles.

SalidaDAO.unidades()

```
public List<Catalogo> unidades() throws SQLException {
    return Sql.catalogo("""
            SELECT c.id_unidad, c.descripcion_unidad FROM viajes.v_catalogo_unidades c
            INNER JOIN viajes.v_unidades_disponibles d ON d.id_unidad = c.id_unidad""");
}
```

- **Administrador**: la orden aparece en "Órdenes de taller", la columna "En taller" de Unidades cambia y "Autobuses" del taller suma 1.
- **Jefe**: la unidad aparece en Flota con estatus de mantenimiento.

Al cerrar la orden pasa lo contrario: el autobús vuelve a ser apto (o queda de baja) y regresa a las listas.

8. Archivos del proyecto Java

8.1 Archivos nuevos

Archivo	Función
model/BusTaller.java	Registro de una unidad en el semáforo: placa, modelo, estatus, apto, último ingreso, observaciones.
model/JefeTaller.java	Jefe de taller activo (para el combo de encargado).

Archivo	Función
dao/jefetaller/TallerJefeDAO.java	Flota, unidades libres, mecánicos del taller y salidas pendientes de una unidad.
controller/jefetaller/FlotaTallerController.java	Pantalla Flota del jefe.
controller/jefetaller/OrdenesJefeController.java	Lista de órdenes del jefe y sus formularios de ingreso y cierre.
controller/admin/AsignacionPersonal.java	Texto de asignación (taller / terminal / taquilla) usado por Personal y Cuentas.
view/jefetaller/barra-jefe-taller.fxml, ordenes-jefe.fxml, flota-taller.fxml	Barra lateral y pantallas del jefe.
view/jefetaller/orden-ingreso-form.fxml, orden-cerrar-form.fxml	Formularios (existían en la carpeta admin; se movieron y se conectaron al controlador nuevo).
fdw_demo.sql (raíz del proyecto)	Script del FDW y sus pruebas. Contiene la contraseña: está en .gitignore.

8.2 Archivos modificados

Archivo	Qué cambió
model/Taller.java	Sin bahías; se agregaron idTaller e idCiudad.
model/OrdenTaller.java	Sin bahía; ahora lleva idOrden, fecha de salida y costo.
model/Empleado.java	Se agregaron idTaller, idTerminal, idTaquilla.
model/Cuenta.java	Se agregó el taller de la cuenta (para el jefe).
dao/admin/TallerDAO.java	Lista real, alta/edición con encargado, baja/reactivación, jefes disponibles, conteo de autobuses.
dao/admin/OrdenTallerDAO.java	Lectura de v_ordenes_taller y llamadas a sp_registrar_ingreso y sp_cerrar_orden.
dao/admin/PersonalDAO.java, CuentaDAO.java, UnidadDAO.java	Leen las columnas nuevas de las vistas; UnidadDAO agrega el estatus de taller.
dao/despachador/SalidaDAO.java	El combo de unidades usa v_unidades_disponibles.
controller/admin/TalleresController.java	Alta, edición, baja, reactivación y encargado.
controller/admin/OrdenesTallerController.java	Auditoría con datos reales; acentos corregidos.
controller/admin/EmpleadoFormController.java	Taller para mecánico y jefe; preselecciona taller, terminal y taquilla al editar.

Archivo	Qué cambió
controller/admin/PersonalListaController.java, CuentasController.java, CuentaFormController.java	Asignación real; "Sede · Taquilla"; taller del jefe.
controller/admin/UnidadesController.java	Columna "En taller".
controller/LoginController.java	JEFE_TALLER entra a ordenes-jefe.
util/Mensaje.java	Muestra el mensaje de la base para Z0005.
App.java, module-info.java	Registran la carpeta de vistas y el paquete de controladores del jefe.
TERMINAL ZACARIASS VERSION MASTER 3.0.txt	Script maestro actualizado (sin bahías, vistas nuevas) para crear la base desde cero.
CAMBIOS Y ASI.txt	Todos los cambios para la base ya creada, como bloques al final.

9. Códigos de error de la base y qué significan

Las reglas de la base lanzan errores con un código propio (SQLSTATE) y un texto claro. La aplicación muestra ese texto tal cual al usuario.

Código	Cuándo	Dónde
Z0005	La unidad está en taller, de baja o inactiva: no se puede asignar a una salida.	Trigger sobre viaje_programado
Z0025	Un jefe de taller no puede tener cuenta sin taller asignado.	Trigger de cuentas
Z0050	El taller no existe o está inactivo.	sp_guardar_taller, sp_cambiar_estado_taller, sp_registrar_ingreso
Z0051	Encargado inválido: no es jefe activo, ya está en otro taller o el encargado actual tiene cuenta.	sp_asignar_encargado
Z0052	El taller tiene personal activo asignado; no se puede desactivar.	sp_cambiar_estado_taller
Z0053	La unidad no existe o está inactiva.	sp_registrar_ingreso
Z0054	La unidad ya tiene una orden abierta.	sp_registrar_ingreso
Z0055	El mecánico debe ser personal activo de ese taller.	sp_registrar_ingreso
Z0056	Tipo de mantenimiento inválido (debe ser preventivo o correctivo).	sp_registrar_ingreso
Z0057	La orden no existe, ya está cerrada o es de otro taller.	sp_cerrar_orden

Código	Cuándo	Dónde
Z0058	Dato inválido: costo negativo, estado final inválido o salida anterior al ingreso.	sp_registrar_ingreso, sp_cerrar_orden
23505	Clave o nombre de taller repetidos.	sp_guardar_taller (restricciones UNIQUE)

10. Cómo probar el flujo completo

10.1 Prueba en la aplicación

1. Entra como administrador. En **Unidades**, la columna "En taller" muestra "OPERATIVO" para cada placa. En **Personal**, crea un empleado con puesto Mecánico y taller "El sapito".
2. Entra como jefe de taller. En **Flota** deben verse todas las placas como operativas.
3. En **Órdenes**, pulsa "+ Registrar ingreso": elige una placa, el mecánico, tipo Correctivo, y registra.
4. En **Flota**, esa placa aparece en mantenimiento correctivo.
5. Entra como despachador. En **Programar salida**, esa placa ya no aparece en el combo de unidades.
6. Entra como administrador. En **Órdenes de taller** aparece la orden; en **Taller**, "El sapito" muestra 1 autobús; en **Unidades**, "En taller" muestra el mantenimiento.
7. Con el jefe: **Cerrar orden** → Operativo. La placa vuelve a Flota como operativa y al combo del despachador.
8. Con el jefe intenta registrar dos ingresos de la misma placa seguidos: la segunda vez debe responder "Esa unidad ya tiene una orden abierta".

10.2 Prueba del trigger en terminal

Necesita al menos una ruta creada. Se hace dentro de una transacción con ROLLBACK para no dejar rastro; el error aparece al intentar el INSERT:

Los números 1, 6 y 2 son ids reales del entorno de pruebas (taller, mecánico, conductor); ajústalos

```
BEGIN;
CALL viajes.sp_registrar_ingreso('DMA20', 1, 6, 3, now(), 'Prueba del trigger', 0, null);
INSERT INTO viajes.viaje_programado (id_ruta, id_estado_viaje, id_unidad, id_conductor, fecha_salida)
VALUES ((SELECT min(id_ruta) FROM viajes.ruta), 1, 1, 2, now() + interval '2 days');
-- Esperado: ERROR  La unidad esta en taller, de baja o inactiva ...
ROLLBACK;
```

10.3 Prueba del FDW en terminal

```
-- en zacabus
SELECT * FROM taller_remoto.bus_taller;                      -- filas del taller
SELECT placa, estatus, apto_para_viajar FROM viajes.v_flota_taller;
-- en taller: cambia un estado y repite la consulta de zacabus: el cambio se ve sin reimportar
UPDATE taller.bus_taller SET id_estado = 3 WHERE placa = 'DMA20';
```

11. Pasar el sistema a Linux, con otras IP, Pgpool y réplica

Esta sección responde: ¿qué se mueve y qué cambia cuando el sistema deje de estar en una sola máquina Windows y pase a las tres máquinas Linux? La respuesta corta: **el diseño, el SQL y el código Java no cambian**. Cambian direcciones, usuarios, permisos de red y la instalación de PostgreSQL y Pgpool.

11.1 Qué NO cambia

- Todas las vistas, procedimientos, triggers y tablas (scripts de CAMBIOS Y ASI.txt, master y TALLER.BD.txt).
- Las tablas foráneas y los nombres de esquema (taller_remoto). Apuntan a srv_taller por nombre, no a una IP.
- Todo el código Java de pantallas y DAO. Ninguna clase contiene una dirección.

Los procedimientos y las vistas se refieren a taller_remoto.bus_taller, no a "la máquina X". La dirección vive en un solo lugar: el servidor foráneo.

11.2 Lo que sí cambia, en cuatro puntos

Punto 1. La dirección del servidor foráneo (una línea)

Mudanza del FDW

```
-- en zacabus, una sola vez, ya con la VM encendida
ALTER SERVER srv_taller OPTIONS (SET host '<IP de la maquina virtual>', SET port '5432');
ALTER SERVER srv_taller OPTIONS (ADD connect_timeout '3');
ALTER USER MAPPING FOR CURRENT_USER SERVER srv_taller
  OPTIONS (SET user 'zaca_fdw', SET password '<contrasena>');
```

Si el mapeo se creó para otro usuario, se puede borrar y crear de nuevo con DROP USER MAPPING / CREATE USER MAPPING. Las vistas y procedimientos no se tocan.

Punto 2. Un rol propio para el FDW, en la VM

En el desarrollo el mapeo usa el superusuario postgres. Entre máquinas eso no debe exponerse: se crea un rol con lo mínimo que el módulo necesita (leer el catálogo y leer/escribir bus_taller y orden_completa):

Permisos mínimos del FDW (orientativo: probar tras crearlo)

```
-- en la base taller (VM)
CREATE ROLE zaca_fdw LOGIN PASSWORD '<contrasena>';
GRANT USAGE ON SCHEMA taller TO zaca_fdw;
GRANT SELECT ON taller.estado_taller TO zaca_fdw;
GRANT SELECT, INSERT, UPDATE ON taller.bus_taller, taller.orden_completa TO zaca_fdw;
GRANT SELECT ON taller.v_semaforo_unidades TO zaca_fdw;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA taller TO zaca_fdw;   -- el numero de orden (identidad)
```

> Estos permisos no se han probado en el entorno del proyecto; son los que se derivan de lo que hacen los procedimientos. Al aplicarlos, ejecutar la prueba completa de la sección 10 para confirmar que no falta ninguno.

Punto 3. Abrir la red hacia la VM

- postgresql.conf de la VM: listen_addresses = '*' (o la IP de la VM).
- pg_hba.conf de la VM: una línea por cada máquina que consultará, por ejemplo host taller zaca_fdw <IP laptop principal>/32 scram-sha-256 y otra igual para la <IP de la réplica>/32. **La réplica también debe poder llegar a la VM**, porque si Pgpool manda una consulta de lectura a la réplica, esa consulta usará su propio FDW.
- Firewall de la VM: abrir el puerto 5432 solo a esas IP (por ejemplo con ufw: ufw allow from <IP> to any port 5432).
- Si la VM está en modo NAT, cambiarla a red en puente (bridge) o reenviar el puerto; si no, las laptops no la alcanzan.
- En Fedora/RHEL: revisar también SELinux y firewalld.

Punto 4. El .env de la aplicación

zaca_bus/.env en cada equipo cliente

```
DB_URL=jdbc:postgresql://<IP de Pgpool>:9999/zacabus
DB_USER=<rol de la aplicacion>
DB_PASSWORD=<contrasena>
```

La aplicación nunca apunta a un nodo concreto, siempre a Pgpool. Es lo único que se modifica del lado Java, y es configuración, no código.

11.3 Pgpool-II y la réplica por streaming

Pgpool-II es el único punto de entrada de la app. Reparte las lecturas entre el principal y la réplica y manda las escrituras al principal. La replicación por streaming mantiene la réplica (laptop 2, puerto 5433) al día.

Tema	Qué hay que saber
Qué replica el streaming	Todo el servidor zacabus (todas sus bases). Incluye el servidor foráneo, el mapeo de usuario y las tablas foráneas, porque viven en el catálogo. Después de una promoción, el FDW sigue configurado igual.
Qué NO replica	La base taller: vive en otro servidor (la VM). Es un punto único de falla; aceptable para el proyecto, pero conviene documentarlo y tener respaldos.
Escrituras	Los CALL a sp_registrar_ingreso, sp_cerrar_orden y demás procedimientos se envían al principal. La réplica es de solo lectura.
Lecturas y retraso	La réplica puede ir unos milisegundos atrasada. Por eso la regla definitiva (trigger Z0005) se evalúa en el principal al escribir, no en la pantalla.
Failover (caída del principal)	Pgpool puede promover la réplica, pero hay que configurarlo (failover_command y health checks) y probarlo. No es automático por sí solo.

Tema	Qué hay que saber
FDW desde la réplica	Una consulta de lectura que pase por la réplica usa el FDW de esa máquina hacia la VM. Por eso la réplica necesita acceso de red a la VM (punto 3).
Misma versión y mismo sistema	La réplica física exige el mismo sistema operativo, la misma arquitectura y la misma versión mayor de PostgreSQL (hoy 16.9) que el principal.

Parámetros típicos de Pgpool a configurar (los nombres exactos dependen de la versión instalada): backend_hostname0/backend_port0 (principal, 5432), backend_hostname1/backend_port1 (réplica, 5433), load_balance_mode, el usuario y verificación de la replicación (sr_check_user), health_check_* y failover_command. Pgpool escucha por defecto en el puerto 9999.

11.4 Pasar a Linux

Tema	Qué hacer
Instalar PostgreSQL 16	En Debian/Ubuntu postgres_fdw viene con el paquete postgresql-16. En Fedora/RHEL, instalar también postgresql-contrib. Después, CREATE EXTENSION postgres_fdw funciona igual que ahora.
Archivos de configuración	En Debian/Ubuntu: /etc/postgresql/16/main/postgresql.conf y pg_hba.conf. Reinicio: systemctl restart postgresql.
Crear las bases	Elegir la collation a propósito (por ejemplo es_MX.UTF-8). Hoy las bases usan Spanish_Mexico.1252 (Windows); el orden alfabético de textos puede variar un poco en Linux. La codificación ya es UTF8, así que los acentos no cambian.
Llevar los datos	Con pg_dump y pg_restore (copia lógica). **No** copiar la carpeta de datos de Windows a Linux.
Orden recomendado para crear la base nueva	1) Script master; 2) bloques de CAMBIOS Y ASI.txt que apliquen o, mejor, un master actualizado; 3) TALLER.BD.txt en la VM + parte A; 4) fdw_demo.sql parte 2 ajustada a la IP real; 5) parte B de CAMBIOS Y ASI.txt.
Java	Instalar JDK 17 (el proyecto compila a Java 17) y las dependencias de JavaFX. El archivo .env se lee de la carpeta desde donde se ejecuta la app. Los nombres de archivos .fxml están en minúsculas, así que no hay problema de mayúsculas en Linux.
Cámara del abordaje	La librería webcam-capture (0.3.12) puede requerir un controlador adicional en Linux para acceder a la cámara. No se ha probado; es lo primero que conviene verificar en una laptop Linux.

11.5 Lista de verificación para la mudanza

1. Instalar PostgreSQL 16 (y postgres_fdw) en las tres máquinas, con la misma versión mayor en las dos laptops.
2. Crear taller en la VM; abrir red y firewall; crear el rol zaca_fdw con sus permisos; correr TALLER.BD.txt y la parte A.
3. Crear zacabus en la laptop principal y restaurar los datos (pg_restore). Ajustar el servidor foráneo con ALTER SERVER y el mapeo de usuario.
4. Correr la parte B (IMPORT FOREIGN SCHEMA debe ejecutarse con la VM encendida y alcanzable).
5. Probar sin Pgpool: .env apuntando directo a la laptop principal, prueba de la sección 10.
6. Montar la réplica (5433) y verificar que replica; abrirle el acceso a la VM.
7. Instalar y configurar Pgpool; cambiar DB_URL a Pgpool y repetir la prueba de la sección 10.
8. Pruebas de falla: apagar la VM (Flota y Órdenes deben mostrar el error en rojo y Programar salida debe fallar con mensaje) y detener el principal (probar el failover).

11.6 Qué podría salir mal

Síntoma	Causa probable
"could not connect to server" al consultar una vista de taller	Red, firewall o pg_hba.conf de la VM; o dirección equivocada en srv_taller.
"password authentication failed" por FDW	El mapeo de usuario tiene un usuario o contraseña que no coinciden con la VM.
"permission denied for table ..." en un ingreso	Falta algún GRANT al rol zaca_fdw (ver el punto 2).
Todo funciona en el principal pero falla en lecturas repartidas	La réplica no puede llegar a la VM, o su pg_hba.conf/mapeo no coincide.
El orden alfabético de listas cambió	Cambio de collation entre Windows y Linux; no es un error.
"column ... does not exist" en una tabla foránea	Se cambió la tabla en taller y no se importó de nuevo.

12. Estado de los scripts y cómo verificarlo

Los cambios para la base ya creada están al final de CAMBIOS Y ASI.txt, uno detrás de otro. Sus encabezados dicen "ESTADO: NO EJECUTADO" porque se escribieron antes de correrlos; según lo confirmado en las pruebas, **ya están aplicados**. Conviene editar esas leyendas para no confundirse.

Bloque (línea aproximada)	Contenido	Notas
"cosas nuevas que no se han ejecutado" (389)	sp_cambiar_estado_taller y un CREATE TABLE administracion.taller	La procedure ya existe. El CREATE TABLE **no se corre nunca**: la tabla ya existe y ese texto aún trae capacidad_bahias.
TALLERES: sin bahías (420)	DROP COLUMN capacidad_bahias; sp_guardar_taller de 5 parámetros; v_talleres; sp_cambiar_estado_taller	Aplicado.
PERSONAL (493)	v_personal con id_terminal e id_taquilla	Aplicado.
CUENTAS (523)	v_cuentas con id_taller y nombre_taller	Aplicado.
FLOTA DEL TALLER (555)	Primera versión de v_flota_taller	**Sustituido** por la versión B3. No volver a correrlo: dejaría la vista con el criterio antiguo ("SIN REGISTRO").
FLUJO DE TALLER, partes A y B (577)	Columna id_estado_ingreso, tablas foráneas, vistas, procedimientos y trigger	Aplicado. La parte A en taller y la B en zacabus.
TALLERES (parte 2) (786)	v_jefes_taller	Aplicado.

12.1 Consultas de solo lectura para comprobar el estado

```
-- conectado a zacabus
SELECT srvname, srvoptions FROM pg_foreign_server;
SELECT foreign_table_schema || '.' || foreign_table_name FROM information_schema.foreign_tables;
SELECT viewname FROM pg_views WHERE viewname IN
  ('v_flota_taller','v_unidades_disponibles','v_mecanicos_taller','v_ordenes_taller',
   'v_talleres','v_talleres_activos','v_jefes_taller');
SELECT proname FROM pg_proc WHERE proname IN
  ('sp_registrar_ingreso','sp_cerrar_orden','sp_guardar_taller','sp_cambiar_estado_taller',
   'sp_asignar_encargado','fn_validar_unidad_taller');
SELECT tgname FROM pg_trigger WHERE tgname = 'tigresito_validar_unidad_taller';

-- conectado a taller
SELECT table_name FROM information_schema.tables WHERE table_schema = 'taller';
SELECT column_name FROM information_schema.columns
 WHERE table_schema = 'taller' AND table_name = 'orden_completa';   -- debe incluir id_estado_ingreso
```

13. Problemas encontrados y cómo se resolvieron

Problema	Causa	Solución
La lista de talleres del formulario de empleado salía vacía.	Solo se mostraba para MECANICO; el DAO era un stub; el taller no se guardaba (null); editar borraba el taller.	Mostrar también para JEFE_TALLER, leer v_talleres_activos, guardar el id y preseleccionarlo al editar.
"Asignación" de Personal siempre vacía.	La columna estaba fija en "—" y v_personal no traía terminal ni taquilla.	Ampliar v_personal y calcular el texto en AsignacionPersonal.
Cuentas mostraba la taquilla pero no la sede.	El texto devolvía solo la taquilla si existía.	Mostrar "Sede · Taquilla".
Dos versiones de sp_guardar_taller.	En PostgreSQL, CREATE OR REPLACE con otros parámetros crea una segunda procedure.	DROP PROCEDURE de la firma vieja antes de crear la nueva.
Insertar órdenes por FDW fallaría con la identidad de id_orden.	postgres_fdw manda NULL en las columnas que no se escriben.	Tabla foránea aparte (orden_ingreso) sin esa columna.
CREATE OR REPLACE VIEW no permitía reordenar columnas.	Solo se pueden agregar columnas al final.	Todas las columnas nuevas se agregaron al final de las vistas.
"error de sintaxis en o cerca de «<»" al probar.	Se pegaron los marcadores <id_orden> tal cual.	Reemplazar los marcadores por ids reales, que se obtienen con una consulta.
Texto con acentos dañados en OrdenesTallerController.	Archivo guardado con codificación equivocada.	Reemplazo de las secuencias dañadas por los caracteres correctos.
Una unidad nueva no aparecería en el semáforo.	El semáforo solo conocía placas insertadas a mano.	LEFT JOIN desde viajes.unidad: sin fila en taller = operativa.
Idea de una segunda conexión Java para el taller.	No era necesaria con FDW.	Se descartó: una sola conexión, los procedimientos viven en zacabus.

14. Pendientes y recomendaciones

14.1 Antes de entregar o desplegar

- Crear el rol zaca_fdw (sección 11.2) y cambiar el mapeo de usuario; hoy usa postgres.
- Agregar connect_timeout al servidor foráneo.
- Actualizar las leyendas "ESTADO: NO EJECUTADO" de CAMBIOS Y ASI.txt y borrar los bloques obsoletos (el CREATE TABLE antiguo de taller y la primera v_flota_taller), o dejarlos marcados.

- Volver a poner TU_PASSWORD en fdw_demo.sql cuando terminen las pruebas (el archivo ya está en .gitignore).
- Preparar un script de despliegue único (un master actualizado) para crear las bases desde cero en Linux.

14.2 Mejoras posibles

- **Placa inmutable**: sp_guardar_unidad permite cambiar la placa; si cambia, el historial en taller (que se relaciona por placa) queda huérfano. Conviene bloquear ese cambio.
- Al registrar un ingreso, la pantalla avisa de las salidas programadas de esa unidad, pero no las reasigna ni las marca (el despacho debe cambiarlas).
- Una unidad "DE BAJA" en taller y una unidad inactiva en el catálogo son dos conceptos separados; se podría sincronizar.
- Segundo FDW (taller → zacabus): opcional. Permitiría que la base taller conozca por sí sola las placas del administrador. Con la conexión única no hace falta.
- La base taller no se replica: si la VM falla, el estado de taller no está disponible. Mantener respaldos (pg_dump) programados.
- Los filtros de ciudad y terminal de Personal y Cuentas siguen marcados como pendientes en el código.
- Seguridad general: la aplicación usa un rol con todos los permisos y hay procedimientos SECURITY DEFINER sin GRANT explícitos; antes de producción conviene un rol de aplicación con permisos mínimos.

15. Glosario

Término	Significado
FDW (Foreign Data Wrapper)	Mecanismo de PostgreSQL para consultar datos de otra fuente como si fueran locales.
postgres_fdw	La extensión de FDW que conecta un PostgreSQL con otro.
Tabla foránea	Tabla local sin datos propios; cada consulta se envía a la base remota.
Servidor foráneo	Ficha con la dirección de la base remota (host, puerto, nombre).
Mapeo de usuario (USER MAPPING)	Credenciales con las que se entra a la base remota.
Semáforo	Estado de cada autobús: apto (verde) o no apto (mantenimiento o de baja).
Pgpool-II	Intermediario entre la aplicación y los servidores PostgreSQL; reparte lecturas y detecta fallas.
Streaming replication	Copia continua de todo el servidor principal hacia una réplica de solo lectura.
Trigger	Regla de la base que se ejecuta sola al insertar o modificar datos.

Término	Significado
SECURITY DEFINER	El procedimiento se ejecuta con los permisos de su dueño, no del que lo llama.
DAO	Clase Java que concentra las consultas a la base de un módulo.